Weekly Report — P0 (Warm-up) and P1 (The dashboard is empty at 08:00)

Data Structures and Management · Assoc. Prof. Dr. Emre Akadal · Fall 2026

Student: Mahmut Eren Akkaymak · Week: 23–29 September 2026 · Repository: github.com/erenkkymak/meydan

1. What I did this week

- Opened the Git repository and fixed my environment for the whole term (see the last line).
- Wrote a data generator (seed 42) and created the orders table at 4 scales: 10k, 100k, 1M and 10M rows. The top 50 sellers hold 38% of the orders (checked: 37.8–38.1%), and cities are weighted by population.
- Wrote a measure() helper: 3 cold runs (each in a new Python process), 5 warm runs, median and min–max, seed and n in every row, plus a growth factor column.
- P0: committed my estimate, then measured the total commission query. P1: committed a new estimate, then measured Q1, Q2 (naive) and Q3 (grouping).

2. P0 — Estimate vs. measurement

Query: SUM(amount × 0.09). I expected about 2 s at 10M rows and a growth of 10× per 10× data. The real result was 0.51 s (warm). So I was about 4× too pessimistic. SQLite does the sum inside its own C loop and the 482 MB file was already in the Windows file cache, so it read about 19.6 million rows per second. The growth shape was correct: 10.4× and 9.7× above 100k rows.

I also found a mistake in my own P0 method: my first “cold” run was only a new connection in the same process, and I ran it only once. That is why cold and warm looked almost the same. In P1 I fixed this with 3 separate fresh processes.

Three numbers from the dossier: (1) commission per order = 640 × 9% = 57.60 TL. But 162 M TL ÷ (42,000 × 365) orders = 10.57 TL, so the dossier numbers do not agree — one “order” looks like about 117 TL, not 640 TL. (2) GMV per active customer = 1.8 bn ÷ 620,000 = 2,903 TL (with 2.4 M registered customers it would be only 750 TL). (3) Campaign day ÷ normal day = 380,000 ÷ 42,000 = 9.05×.

3. P1 — Results

Q2 asks: how many customers bought from two different sellers on the same day? Q2 compares every order with every other order. Q3 answers the same question with GROUP BY. Both return one number (distinct customers). Warm medians:

Query	n	Time (s)	Growth factor	Customers found
Q1 commission	10,000,000	0.517	10.1× per 10×	—
Q2 naive	1,250 → 10,000	0.039 → 2.41	3.9–4.0× per 2×	0 / 4 / 22 / 86
Q3 GROUP BY	10,000	0.0033	—	86
Q3 GROUP BY	10,000,000	5.62	10.8–13.2× per 10×	82,751

Key points: Q2 grows about 4× every time the data doubles, so its time grows with n^2 (measured exponent 1.98). Q3 gives exactly the same answer as Q2 at every size (0, 4, 22, 86), and at 10,000 rows it is about 730× faster. I did not run Q2 on 100k and 1M rows; from the pattern it would take about 4 minutes and about 6.2 hours. I switched off SQLite’s automatic index, because otherwise the “naive” query is secretly not naive.

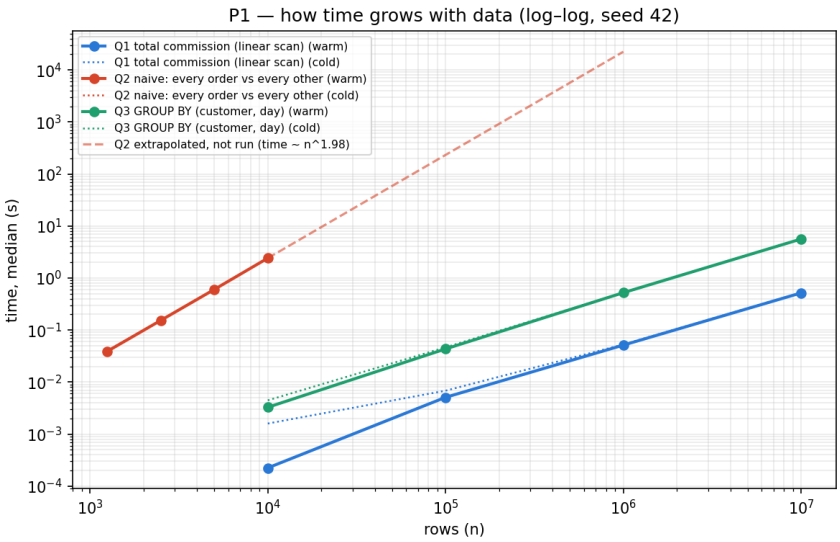


Figure 1. Rows vs. time, log–log scale. Q1 and Q3 have slope ≈ 1, Q2 has slope ≈ 2.

4. My answer to Sinem (CFO)

No, I would not approve the bigger server. The number behind this is Q2's growth factor: 2× data gives 4× time. A 2× server can only halve the time one time, but data grows 60% every year.

- In 12 months, if time grows like the data: 11 × 1.6 = 17.6 h (8.8 h with the big server). If it grows like Q2: 11 × 2.56 = 28.2 h (14.1 h with the big server). Both are still too late for 08:00.
- The empty dashboard costs 40 × 2 h × 350 TL = 28,000 TL per day, about 616,000 TL per month. The server costs 95,000 TL per month. This makes the problem urgent, but the real fix is to find and rewrite the slow step. A one-month server only makes sense as a short bridge.
- Assumption: the real ETL has a step like Q2. The case supports this: data grew about 40% in 9 months but the job time grew 83%.

5. What I learned and what went wrong

- The growth shape (1× or n^2) was more important for the decision than the exact seconds. My P1 estimates had the right shapes but were 20–40% off in seconds.
- A "cold" run must really be cold, and one run is not a measurement.
- My first explanation for the 21× jump from 10k to 100k in P0 was backwards. The real reason: the 10k table fits in SQLite's 2 MB page cache, the 100k table does not.
- The measurements must run on my Windows machine, not on another environment, so I ran the benchmark with a .bat file on the same PC.

6. AI use

I used an AI model to write the generator, the measure() helper and the queries, and to review my report. It was wrong in three places: its first "naive" query would have used an automatic index, it first counted order pairs instead of distinct customers, and it explained the 10k→100k jump in the wrong direction. I found these with EXPLAIN QUERY PLAN, by checking that Q2 = Q3, and by checking the direction of the numbers.

7. Next week

- Push the repository to GitHub under the name meydan and post the link to Classroom.
- Prepare my budget table (400,000 TL / 30 engineer-days) so I can use it in the next case.

Environment: AMD Ryzen 9 9950X3D (16C/32T), 96 GB DDR5, NVMe SSD · Windows 11 Pro 10.0.26200 · Python 3.11.9 · SQLite 3.45.1 (sqlite3 module 2.6.0) · matplotlib for the chart · seed 42